

# Predicting Players in Motion

NFL Big Data Bowl 2026, Prediction track • Neel Ahuja

[Code on GitHub](#) • [Kaggle submission notebook](#)

Twenty-two players are tracked at 10 frames a second. Given everything up to the throw, the task is to predict where the targeted players will be for each frame until the ball lands (sometimes forty-plus frames ahead), scored by RMSE on the  $(x, y)$  coordinates over roughly 14,000 plays. The catch that shapes the whole project: training is the 2023 season while the hidden test is a later one, so a model that memorizes 2023 does not travel. A constant-velocity baseline scores 1.645 and sets the floor.

## The model

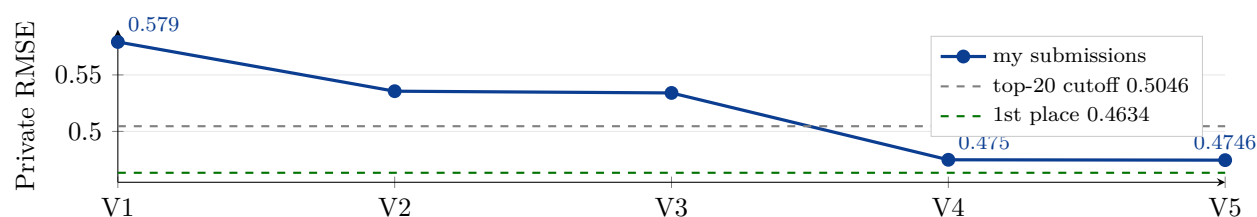
A compact CNN+Transformer, roughly 1.4M parameters early on and 4.9M at full size ([train.py](#)). Each of the 22 players becomes one token: a depthwise 1-D CNN reads that player's last 20 frames on its own (position, speed, acceleration, the sine and cosine of heading and orientation, and vectors to the ball's landing spot and the intended receiver). A pre-norm SwiGLU Transformer then lets the tokens attend across each other, since a defender's path is largely a response to the receiver's, and a convolutional head decodes each into a per-frame displacement plus its own variance out to 48 frames. Training uses a Gaussian negative-log-likelihood (which penalizes being both wrong and overconfident) with velocity and acceleration auxiliary heads, heavy geometric augmentation (random rotation, mirroring, early starts), EMA weight averaging, and a horizontal-flip test-time average.

## Autoresearch: turning tuning into an experiment loop

Rather than hand-tune, I ran the project as a Karpathy-style autoresearch loop, a discipline about not fooling yourself. [prepare.py](#) is frozen: it builds the data and defines the RMSE metric and the validation split, so nothing I optimize can quietly change how it is judged. [train.py](#) is the only file I edit, each run gets the same fixed wall-clock budget so two experiments are comparable, and every result is re-scored from saved predictions and logged with its git commit to [results.tsv](#). Twelve overnight hours produced 33 tracked experiments, and the biggest lever it found was that the published recipe over-weights the velocity/acceleration loss for my training budget: cutting that weight while raising the learning rate moved the model from 0.94 to 0.63.

## From holdout to leaderboard

My champion scored 0.478 on held-out weeks but 0.579 on the real test, because the holdout was later weeks of the same 2023 season while the test is a new one. So I pointed the loop at the leaderboard itself and swept learning rate, model width and depth, the auxiliary-loss weight, augmentation, the decoder, and the ensemble makeup. Scaling to full size and 37k steps took a single model to 0.536, and fold ensembling to 0.534. The largest jump was not a hyperparameter: a parity test replayed raw plays through the submission path and the local scorer, they disagreed by 0.05, and the diff showed one feature (frames-to-predict) was zeroed for every unscored player at inference. Fixing that one line dropped the identical ensemble to 0.475.



Private RMSE across the five submitted versions; the drop at V4 is the inference-bug fix. Lower is better.

## The final model

The submission is a four-way ensemble of the full network, each member 4.9M parameters (64 channels per feature, a 256-wide 3-layer Transformer) trained on a different 2023 fold. Three members integrate per-frame displacements and one uses a direct head, which decorrelates their errors. The private RMSE is 0.47458, which lands 4th of roughly 1,900 teams (behind only 0.4634, 0.4648, and 0.4658) and well inside the top-20 cutoff of 0.5046. The gap left to first is single-model quality, not ensembling.